MMAUG / AI & DEVOPS BOOTCAMP

From Vague Idea to Structured Spec: Your First Agentic AI Workflow

A human-in-the-loop pattern for working with AI agents

Roberto Corella / AI Evangelist · Microsoft MVP · MCT

24 October 2026

Scan to join the Livestream

A Global Community for AI, Cloud, Infra, DevOps and Tech Enthusiasts

Learn. Innovate. Build.

mmaug.com

Meet the speaker

Who is talking to you today



Roberto Corella

AI Evangelist · Microsoft MVP · MCT

I help people get reliable results from AI agents, whatever tool they use.

- Microsoft MVP and Microsoft Certified Trainer (MCT)
- Creator of the Aprende Business Central YouTube channel and blog
- Today: a way of working with AI agents that holds up with any tool

CONNECT

Blog rcorella.github.io/LearningBusinessCentral
LinkedIn linkedin.com/in/robertocorella

YOUTUBE CHANNEL
Aprende Business Central

Today’s session

One vague request, followed from a two-line email to a working app

01	**The request**	Why a two-line email cannot be built yet
02	**The pattern**	Propose → Review → Approve, at every stage
03	**Functional spec**	Instruct, propose, review and freeze v1.0
04	**Technical spec**	Hand over context, then check traceability
05	**Build and check**	Small slices, a checklist and the failure modes

The promise vs. the reality

Inside the box, the agent makes dozens of decisions you never see

THE PROMISE

- **“The agent does everything”**
- You ask, it delivers
- When it guesses right, it feels like magic

THE REALITY

- **It predicts plausible output**
- It fills every gap with a guess
- It sounds equally confident when right or wrong

QUESTION **Who is accountable for the result? Only the human knows what was meant.**

The request

Friendly, urgent, and it sounds clear. Could you start building it today?

TIMEBOX

1

MINUTE

WORK MODE

Pair with your neighbour

PREREQUISITE None

YOUR TASK

What would you ask Sam before building anything?

From: Sam (Operations)

Subject: App for our equipment

“Hi, we keep losing track of who has our laptops and stuff. Can you make us an app to track it? Would be great to have it soon. Thanks!”

SUCCESS CHECK

Each pair has at least three questions for Sam

Two questions to ask Sam first

Each answer removes a guess the agent would otherwise make on its own

01 / WHAT

“What exactly do you need to track?”

- Only laptops, or also monitors, phones and cables?
- How many items, and does each one have an asset tag?

Sam: laptops, monitors, phones and cables; about 80 items, each with an asset tag

02 / WHO + RULES

“Who records a loan, and what happens if an item comes back late or damaged?”

- Do employees log loans themselves, or an admin?
- Maximum loan time? Reminders? Lost or damaged items?

Sam: 2 admins record every loan; 14 days maximum; admin flags lost items

Bonus question: *“What does ‘soon’ mean for you?”* Sam: a first usable version in 4 weeks

A dozen questions hiding in one email

Every unanswered question is a guess waiting to happen

01

WHO

Who borrows? Who approves? Who sees what?

02

WHAT

Only laptops, or cables and phones too? Unique items or counted by quantity?

03

HOW

How does a loan start and end? Who records it: borrower or admin?

04

RULES

Maximum loan time? What if an item is late, lost or damaged?

05

WHERE

Web, mobile or both? Replace the Excel sheet? Import existing data?

06

WHEN

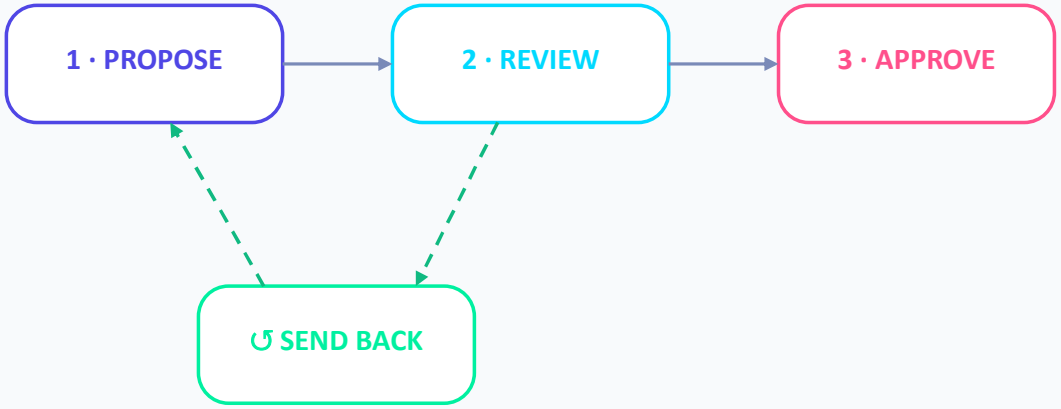
What does “soon” mean: a first version in two weeks, or a full product?

Propose → Review → Approve

The whole method fits on a sticky note, and it applies at every stage

Agent proposes, human decides

The agent drafts fast; only you know what Sam really needs. **Nothing moves on unapproved**, and every approval leaves a trace: version, date, decisions.



WHY AT EVERY STAGE

A wrong assumption caught in the requirements costs minutes. Caught in running code, it costs days.

STAGE / 01

The functional specification

> *Instruct the agent, read what it proposes, review it, and freeze v1.0*

Instruct: ask before you write

A good instruction is mostly rules about behaviour, not about content

functional-spec-prompt.txt

01 **ROLE** Business analyst writing for non-technical readers
02 **INPUT** Sam's email, pasted verbatim
03 **RULE 1** Ask clarifying questions first; no spec until answered
04 **RULE 2** Tag every statement CONFIRMED or ASSUMPTION
05 **RULE 3** No unrequested features; park ideas as out of scope
06 **RULE 4** Acceptance criteria answerable with yes or no
07 **STRUCTURE** Purpose, roles, scope, stories, rules, criteria
08 **OUTPUT** One self-contained HTML file

1. **Rule 1 matters most.** Permission to say "I don't know yet" turns an app into questions.

2. **Tags make hidden decisions visible.** Every ASSUMPTION is something to confirm with Sam.

3. **Expect about 10 questions.** Answer them as Sam would; the full prompt is in the handout.

Propose: the specification as an HTML page

A spec nobody reads is worth nothing, so presentation matters

WHY HTML

Opens in any browser, shows gaps at a glance, and prints to PDF

1. Version number and a DRAFT badge
2. Summary a manager reads in 20 seconds
3. Numbered rules BR-01 and criteria AC-01
4. **ASSUMPTION** in amber, **open questions** in red

SCREENSHOT OF THE RENDERED SPEC
functional-spec-v0.1.html

Fallback: pre-generated spec open in a browser tab

Review: human-in-the-loop gate 1

A helpful guess looks exactly like a fact, so ask where each claim came from

ASK OF EVERY SECTION

Complete?
Anything missing from Sam's answers?

Faithful?
Anything nobody asked for?

Testable?
Could two people agree it passes?

Consistent?
Do numbers and terms match?

Where did that come from?
Every unlabelled claim

WHAT THE AGENT DID	FAILURE MODE	FIX
Added reservations and QR-code scanning	**Scope creep**	Move to "Out of scope"
"Users sign in with a Microsoft account", unlabelled	**Silent assumption**	Confirm with Sam, then tag CONFIRMED
"The app must be fast and easy to use"	**Untestable criterion**	"Recording a loan takes at most 3 steps"
Loan period 14 days in one section, 30 in another	**Inconsistency**	Align to 14 days
Nothing on returning an item that is not on loan	**Missing edge case**	Add business rules and criteria

Send it back in one message: *"Apply these five changes. List exactly what you changed. Change nothing else."*

Approve: freeze version 1.0

Approving is a deliberate act, not a nod

STATUS

DRAFT → APPROVED · v1.0

- The decisions log records **what changed and who decided**
- An open point is fine if it is **visible and has an owner** (D-02)
- After approval, changes need a **change request**, not a quiet edit

DECISIONS LOG

ID	DECISION	DECIDED BY	DATE
D-01	Reservations and QR codes are out of scope for v1	Sam	Today
D-02	Sign-in method to be confirmed with IT	Sam / IT	**Pending**
D-03	Loan limit is 14 days	Sam	Today

Saved as **functional-spec-v1.0.html**

STAGE / 02

Handoff and technical specification

> *Context gets lost between stages, so the handoff is a deliverable too*

The handoff brief

The next chat, agent or person only knows what we hand over

GOAL	**The goal in three sentences**	functional-spec-v1.0
DECISIONS	**Decisions log, open points and owners**	D-01 to D-03
OUT OF SCOPE	**Stated explicitly, so nobody adds it back**	Reservations, QR codes
NON-NEGOTIABLE	**Rules that must not change**	BR-03: 14-day limit

IDs carry over: BR-xx and AC-xx keep the same number in every later document

CONSTRAINTS

Only a human can set these

- Technology the team can maintain
- Hosting
- Budget
- Security rules

The agent leaves this section empty; you fill it in.

Review the technical spec: gate 2

You do not judge the database design; you check the traceability table

TRACEABILITY TABLE

REQUIREMENT	TECHNICAL ELEMENT	TEST
BR-01 One open loan per item	Reject a loan if an open loan exists	T-01
BR-03 Maximum loan 14 days	Due date set automatically at loan start	T-03
AC-05 Reminder 2 days before due	**Missing: dropped between documents**	—
AC-07 Admin flags a lost item	Status “Lost” hides it from the list	T-07
No requirement	**Real-time analytics dashboard**	—

REVIEW QUESTIONS

Covered? Does every BR and AC map to something?

Traced? Is anything here that no requirement asked for?

Fits? Do the technology choices fit our constraints?

Right size? 25 users and 80 items, not millions

Same pattern: *“Fix these two points. List what you changed. Change nothing else.” Then approve technical-spec-v1.0.*

STAGE / 03

Implementation

> *Small slices, each one proposed, reviewed and approved*

Propose: build in small slices

First the agent proposes a build plan; the human approves it before any code

BUILD PLAN · ONE SLICE AT A TIME

SLICE	DELIVERS	COVERS
1	Item list, add item, import from Excel	BR-01, AC-01, AC-02
2	Start a loan; due date set automatically	BR-03, AC-03
3	Return an item	AC-04
4	Reminder two days before the due date	AC-05
5	Flag an item as lost or damaged	AC-07

RULES FOR THE AGENT

- **One slice at a time;** stop and show the result
- **Name the requirement IDs** in the code and the tests
- **Say what you ran and what you saw,** not just “done”
- **Touch nothing** outside the current slice

The rhythm: plan → approve → slice → try it → approve → next slice. Works with a chat assistant, a low-code platform or a coding agent.

Review the app: gate 3

You do not need to read the code. You need to use the app.

HOW TO REVIEW

- **Walk each acceptance criterion** as a real user
- **Try a bad day:** wrong inputs, double clicks, an item already on loan
- **Compare the agent's report** with what you actually saw

When the agent says "done", your next words are "show me".

WHAT THE AGENT DID	FAILURE MODE	HOW YOU CATCH IT
Works with perfect input only	**Happy path only**	Return an item that is not on loan
14-day limit typed in five places	**Hard-coded value**	Ask: "Where is this rule defined?"
"All tests pass"	**Tests never run**	Ask for the run output; break something on purpose
Changed the item screen during the loan slice	**Out-of-slice change**	Ask for every file or screen touched
Slice 4 "done", but no reminder arrives	**Claim without evidence**	Trigger it yourself, or ask for proof

Result: a tick-list of acceptance criteria, each marked passed, failed or not tested

Three gates caught nine agent mistakes

From Sam's email to a working app, with a human review between every stage



Each stage closes with a human review gate: nothing moves on until it is approved

Checklist: before you trust an agent's output

Nine questions, none of them technical. They work on any agent output.

01 SOURCE
Did I give it the original request, not my summary?

02 QUESTIONS
Did it ask before assuming? Is every assumption labelled?

03 FAITHFUL
Is there anything here nobody asked for?

04 COMPLETE
Is every requirement covered? Any blank cells?

05 TESTABLE
Can each criterion be answered yes or no?

06 CONSISTENT
Do numbers and terms match everywhere?

07 EVIDENCE
Did it show a result, not only a claim?

08 SCOPE
Did it change only what I asked? Do I have the list?

09 APPROVAL
Which version did I approve, who approved it, and when?

Questions 2 and 7 catch the most mistakes: *"Where did that come from?" and "Show me."*

Common failure modes, and what catches them

The mistakes a tired human makes too, only fluent and without hesitation

FAILURE MODE	TYPICAL SIGN	WHAT CATCHES IT
Silent assumption	A confident detail nobody provided	Ask “where did this come from?”; require ASSUMPTION labels
Scope creep	Extra features that sound useful	Compare with the approved source; park ideas as out of scope
Dropped requirement	In stage 1, gone in stage 2	Traceability table with IDs
Vague criteria	Words like fast, easy, intuitive	Ask “how would I check this?”
Inconsistency	Two numbers for the same rule	Search the document for the value
Happy path only	Works until something unexpected happens	Try a bad day on purpose
Claim without evidence	“Done”, “all tests pass”	Ask “show me”

Every counter-move is a human question. That is why the human stays in the loop.

Takeaways

Then tell me what the agent got wrong: that is where you will learn the most

01

01

Pattern

Propose → Review → Approve, at every stage

02

02

Checklist

Nine questions before you trust any agent output

03

03

Mindset

The agent is a fast, tireless assistant; you own the intent

04

04

This week

Run one small, real request from your inbox through the pattern

Handout with the prompts and checklist: [add QR code or short link]

THANK YOU.

Questions? Pick one small request this week and run it through the pattern.

WEBSITE
mmaug.com

MEETUP
Malta Microsoft AI User Group

SPEAKER
Roberto Corella · msdyn365bc.blogspot.com